



Session 13

CI/CD Pipelines: GitHub Actions

GitHub Actions · Quality Gates · Docker Build & Push · Notification Service (Full)



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Wednesday

TIME

3:00 – 5:30 PM

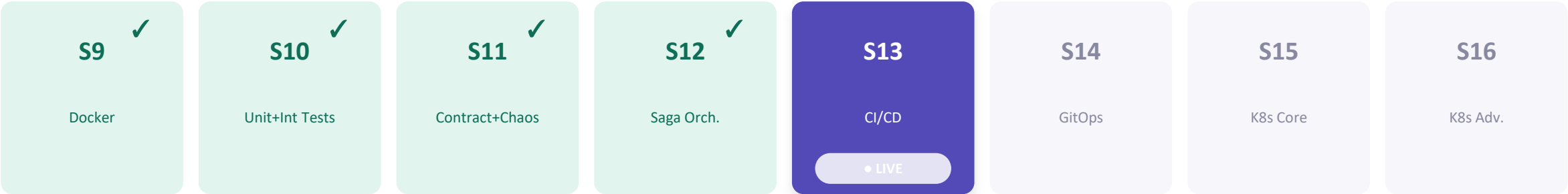
DURATION

2.5 Hours

PHASE

**Phase 2 — Quality &
Deployment**

Where We Are



Sessions 9-12 shipped a containerized, tested, orchestration-ready platform. Session 13 automates the path from commit to registry.

Full 29-session roadmap is in the Kickoff deck only.

Two Ingredients Are Ready. Today We Automate Them.



S9 — Docker

Every core service has a multi-stage Dockerfile.
docker compose up -d starts the platform reproducibly.



S10 + S11 — Tests

Unit, slice, integration (TestContainers),
contract (Pact) and chaos-validated resilience.

What's still manual: a developer builds the JAR, copies it to a server, and hopes the environment matches. Today that stops.

THE PROBLEM

Friday, 4 PM. Ready to Ship.

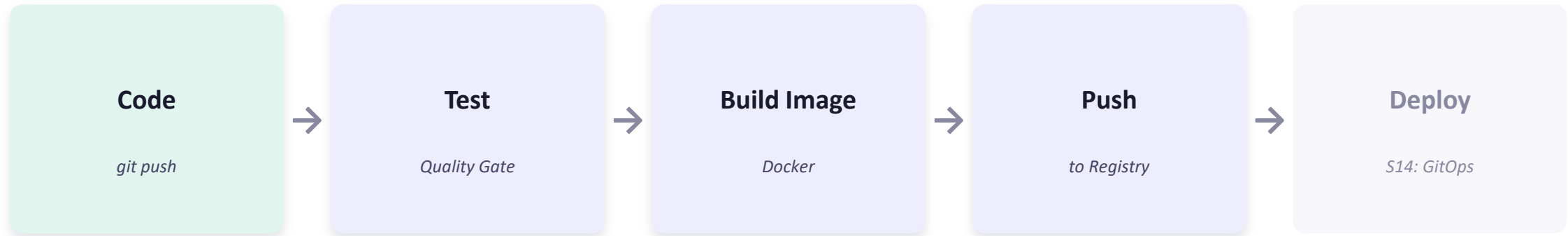
- 1 Ahmed builds the JAR locally: `mvn package -DskipTests` — skipping tests, “it works on his machine.”
- 2 He SSHs into the server. Copies the JAR. Stops the old service. Starts the new one.
- 3 Error: `ClassCastException`. The server runs JDK 17. Ahmed's laptop runs JDK 21.
- 4 5 PM. Rollback: re-SSH, find the old JAR, restart. 45 minutes of stress.
- 5 **The team quietly stops shipping on Fridays. Features pile up — releases grow bigger and scarier.**

The real question: what is the cost of FEAR?

CI/CD Breaks the Fear Cycle

	Manual Deployment	CI/CD Pipeline
Who builds?	Developer (locally)	Runner (VM in GitHub)
Environment	Developer's laptop — may differ from server	Controlled, reproducible runner image
Tests run?	“Usually” — skipped when rushed	Always — pipeline fails if tests fail
Docker image?	Built manually, tagged inconsistently	Tagged with git SHA — traceable
Rollback?	SSH back, find old binary, restart	Re-run previous pipeline → redeploy old image
Release frequency?	Weekly / biweekly (fear-driven)	Multiple per day (confidence-driven)

The Pipeline: Code to Registry



Session 13 builds stages 1–4. Deployment (stage 5) is GitOps — Session 14.

GitHub Actions — Workflow Anatomy

on :

The trigger — when does this run?

push to main, pull_request, schedule, workflow_dispatch

jobs :

A collection of steps on one runner

Jobs run in parallel by default — use needs: to order them

steps :

Ordered commands within a job

uses: a packaged action · run: a raw shell command

runs-on :

The runner (VM/container)

ubuntu-latest — GitHub-hosted, or self-hosted

test Job — The Quality Gate

```
# .github/workflows/product-service-ci.yml
# Location: in the REPOSITORY ROOT
```

```
name: Product Service CI                                # displayed in GitHub Actions UI

on:                                                       # TRIGGER: when does this run?
  push:
    branches: [main]                                     # on push to main branch
    paths:                                               # OPTIMIZATION: only run if these files changed
      - 'services/product-service/**'                  # ignore changes to other services
  pull_request:
    branches: [main]                                     # also run on PRs to main (for review safety)

env:                                                       # SHARED environment variables for all jobs
  JAVA_VERSION: '21'
  REGISTRY: ghcr.io                                     # GitHub Container Registry
  IMAGE_NAME: ${GITHUB_REPOSITORY}/product-service
```

 **paths filter:** Without paths: [...], every push to main triggers CI for ALL services. With it, only changes to product-service files trigger this workflow. Create separate workflows per service — or use a matrix strategy (shown below).

test Job — The Quality Gate — Continue

```
# .github/workflows/product-service-ci.yml
# Location: in the REPOSITORY ROOT
```

```
jobs:
  test:                                # JOB 1: Quality Gate
    name: Test & Quality Gate
    runs-on: ubuntu-latest              # RUNNER: GitHub-hosted Ubuntu VM
    steps:
      - name: Checkout code
        uses: actions/checkout@v4       # ACTION: clone the repository

      - name: Set up JDK ${ env.JAVA_VERSION }
        uses: actions/setup-java@v4
        with:
          java-version: ${ env.JAVA_VERSION }
          distribution: 'temurin'        # Eclipse Temurin (same as our Dockerfile)
          cache: maven                   # OPTIMIZATION: cache ~/.m2 between runs

      - name: Run tests (Quality Gate)
        run: mvn test -pl services/product-service -B # -B = batch mode (no ANSI colours)
        # ← If ANY test fails, this step fails → the entire job fails → build stops
        # ← This is the Quality Gate: no broken code reaches the Docker build step
```

fail-fast vs allow-failure in Quality Gates

This is the key tactical decision in CI pipeline design. Present it before adding the Docker build step.

Option A — fail-fast: true (default, recommended):

If the test job fails, the Docker build job does NOT start. Pipeline stops immediately.

- ☐ Fastest feedback: developer knows within 3 minutes that tests are broken.
- ☐ No broken Docker image is ever pushed to the registry.
- ☐ Matrix builds: if Java 21 fails, Java 17 test run is also cancelled (may be acceptable).

Option B — continue-on-error: true (for quality REPORTS, not gates):

Test failures do not stop the pipeline — results are recorded as warnings.

- ☐ Useful for: Checkstyle, SpotBugs, coverage reports — informational, not blocking.
- ☐ If used on unit tests: broken code reaches Docker build. Defeats the Quality Gate purpose.

Our decision: mvn test = fail-fast (blocking). Checkstyle/SpotBugs = continue-on-error (reporting).

fail-fast vs allow-failure in Quality Gates

Critical service, failure must block deployment?

→ **fail-fast (default)**

Optional check the team reviews manually?

→ **allow-failure (continue-on-error)**

Test coverage threshold breached?

→ **fail-fast**

Experimental linting rule, still tuning?

→ **allow-failure — informational only**

Never use continue-on-error on the test step — “tests passing” must mean what it says.

Push Broken Code → Quality Gate Fails

✓ Checkout code

✓ Set up JDK 21

✗ Run tests (Quality Gate)

⦿ Build & Push Docker Image — SKIPPED (needs: test failed)

A broken test never reaches the Docker build step. The pipeline is the safety net.

Where Session 13 Stops

SESSION 13 — CI

← STOPS HERE

Code → Test (Quality Gate) → Build Image → Push to Registry



SESSION 14 — GITOPS

Registry Image → Update Git Manifest → ArgoCD Sync → Kubernetes Cluster

The pipeline produces an artifact — a Docker image with a SHA tag. GitOps decides when and how it reaches the cluster.

build-and-push Job — Setup (Part 1)

```
build-and-push:                                     # JOB 2: Docker build + push
  name: Build & Push Docker Image
  runs-on: ubuntu-latest
  needs: test                                       # DEPENDENCY: only runs if 'test' job passes
                                                    # needs: test → fail-fast enforced here

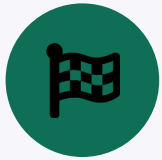
  permissions:
    contents: read
    packages: write                                # required to push to GHCR

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3           # Buildx: enables multi-platform builds and build cache

    - name: Login to Container Registry
      uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN } # auto-injected — no manual secret needed for GHCR
        # For DockerHub: use secrets.DOCKER_USERNAME + secrets.DOCKER_TOKEN instead
```

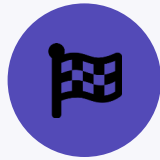
Docker Image Tagging Strategy



sha- prefix

`sha-abc1234`

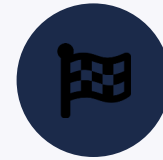
Traceable to the exact commit. Every image can be tied back to its source.



latest

`latest`

Convenience tag for the newest main-branch build. Enabled only on push to main.



branch ref

`main`

Tracks the branch name. Useful for dev/staging environments.

Rule: tag with sha- for deployment traceability. latest is convenience only — never deploy from it in production.

build-and-push Job — Metadata & Build (Part 2)

```
- name: Extract Docker metadata (tags, labels)
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
    tags: |
      type=ref,event=branch          # tag: main
      type=sha,prefix=sha-           # tag: sha-abc1234 (traceable to commit)
      type=raw,value=latest,enable=${{ github.ref == 'refs/heads/main' }}

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: services/product-service # path to Dockerfile
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    cache-from: type=gha              # use GitHub Actions cache for faster builds
    cache-to: type=gha,mode=max
    # NOTE: DEV ONLY — In production: add --platform linux/amd64,linux/arm64 for multi-arch
```

Quality Gate Passes → Image Pushed to GHCR

- ✓ `test` — Quality Gate
- ✓ `build-and-push` — Docker Buildx + Login
- ✓ Image pushed: `ghcr.io/org/product-service:sha-abc1234`
- ✓ Image pushed: `ghcr.io/org/product-service:latest`

Both images are now visible in GitHub Packages — traceable, versioned, ready for GitOps in Session 14.

Notification Service: From Bonus to Production

S7 Bonus (homework)

- ✗ @KafkaListener → log.info("Email sent")
- ✗ No retry
- ✗ No dead-letter queue
- ✗ Failure = silent event loss

S13 Full Implementation

- ✓ @RetryableTopic — 3 attempts, exponential backoff
- ✓ Dead Letter Topic (.DLT) for exhausted retries
- ✓ Actuator health endpoint — Kafka status
- ✓ Registered in Eureka + docker-compose + CI

Notification Service — dependencies

`pom.xml` — `notification-service`

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

NotificationService.java — @RetryableTopic and DLT

NotificationService.java — notification-service

```
package com.microservices.pro.notification;

@Service @Slf4j
public class NotificationService {
    // — PAYMENT COMPLETED: send order confirmation email —————
    @RetryableTopic(
        attempts = "3",                // try 3 times before giving up
        backoff = @Backoff(delay = 1000, multiplier = 2.0), // 1s → 2s → 4s
        dltTopicSuffix = ".DLT",        // failed events → payment-events.DLT
        autoCreateTopics = "false"      // topics must exist — do not create on-the-fly
    )
    @KafkaListener(topics = "payment-events", groupId = "notification-service")
    public void handlePaymentCompleted(String rawEvent) {
        if (!rawEvent.contains("PaymentCompleted")) return;
        var event = parsePaymentCompleted(rawEvent);
        log.info("[NOTIFICATION] Sending confirmation email for order: {}", event.orderId());
        // In production: call email provider (SES, SendGrid, etc.)
        // For the course: simulate email sending with a log + configurable delay
        sendConfirmationEmail(event.orderId(), event.transactionId());
        log.info("[NOTIFICATION] □ Confirmation sent for order: {}", event.orderId());
    }
}
```

NotificationService.java — @RetryableTopic and DLT — Continue

NotificationService.java — notification-service

```
// — ORDER CANCELLED: send cancellation notification —————
@RetryableTopic(attempts = "3",
    backoff = @Backoff(delay = 1000, multiplier = 2.0),
    dltTopicSuffix = ".DLT")
@KafkaListener(topics = "inventory-events", groupId = "notification-service-cancel")
public void handleOrderCancelled(String rawEvent) {

    if (!rawEvent.contains("OrderCancelled")) return;

    OrderCancelledEvent event = parseOrderCancelled(rawEvent);

    log.info("[NOTIFICATION] Sending cancellation notification for order: {}", event.orderId());

    sendCancellationNotification(event.orderId(), event.reason());
}
```

NotificationService.java — @RetryableTopic and DLT — Continue

NotificationService.java — notification-service

```
// — DLT HANDLER: log events that exhausted all retries —————
@dltHandler
public void handleDlt(String rawEvent,
    @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    log.error("[NOTIFICATION] □ DLT: Event from topic '{}'' exhausted all retries. "+
        "Manual intervention required. Event: {}", topic, rawEvent);
    // In production: send to monitoring/alerting (PagerDuty, Slack, etc.)
}

private void sendConfirmationEmail(String orderId, String txId) {
    log.info("[EMAIL] Order {} confirmed. Transaction: {}", orderId, txId);
    // DEV ONLY: simulate sending. Production: use JavaMailSender or email SDK.
}

private void sendCancellationNotification(String orderId, String reason) {
    log.info("[EMAIL] Order {} cancelled. Reason: {}", orderId, reason);
}
}
```

DLT Flow — Happy Path vs Retry Exhaustion

HAPPY PATH

`PaymentCompleted` event → `handlePaymentCompleted()` → ✓ email sent, no retry needed

RETRY EXHAUSTION

`Attempt 1 fails` → `wait 1s` → `Attempt 2 fails` → `wait 2s` → `Attempt 3 fails` → `wait 4s`

→ `@DltHandler` → `payment-events.DLT` → `log.error("Manual intervention required")`

Failed events are never silently dropped — they land in the Dead Letter Topic for investigation or manual replay.

NotificationService.java — application.yml

application.yml — notification-service

```
server:
  port: 8085

spring:
  application:
    name: notification-service
  config:
    import: optional:configserver:http://localhost:8888
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
```

NotificationService.java — application.yml — Continue

`application.yml` — `notification-service`

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

management:
  endpoints:
    web:
      exposure:
        include: health,info,kafka
  health:
    kafka:
      enabled: true # shows Kafka consumer status in /actuator/health
```

Add to docker-compose.yml

docker-compose.yml

```
notification-service:
  build: ./services/notification-service
  image: notification-service:local
  ports: ["8085:8085"]
  depends_on:
    eureka-server:
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    - SPRING_CONFIG_IMPORT=optional:configserver:http://config-server:8888
    - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
    - SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:9092
  networks: [platform-net]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8085/actuator/health"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Lab 11A — CI Pipeline for Platform Services

- 1 Create `.github/workflows/product-service-ci.yml` — test + build-and-push jobs
- 2 Implement notification-service: `@RetryableTopic` + `@DltHandler` + docker-compose entry
- 3 Register NOTIFICATION-SERVICE in Eureka; verify it appears in the dashboard

Acceptance Criteria

- Broken test push → pipeline shows  , build-and-push does NOT run
- Fixed code → image appears in GHCR tagged sha- and latest
- `@RetryableTopic`: 3 attempts, exponential backoff configured
- `@DltHandler` present and logs DLT events

What We Added Today

🔗 Capability Added

Automated Quality Gates & Continuous Delivery

Every push to main is automatically tested and packaged into a traceable Docker image.

🔗 Services Updated

NEW: notification-service (port 8085) 🔗

@RetryableTopic + @DltHandler · .github/workflows/*.yml added

Enterprise E-Commerce Platform — After Session 13: CI pipeline (GitHub Actions) + Notification Service officially complete

Config



Eureka



Gateway



Product



Order



Payment



Inventory



Notif.



All 8 services now 🔗 complete — Notification Service moved from 🔗 to 🔗 this session.

5 Errors You Will Hit — And How to Fix Them

“permission denied” pushing to GHCR

Add permissions: packages: write to the build-and-push job.

Maven cache not working, downloads every run

Ensure cache: maven is set AND pom.xml is committed.

paths filter ignores all pushes

Use `**` for recursive matching: `services/product-service/**`

“No method annotated with @DltHandler found”

@DltHandler must be in the SAME class as @KafkaListener.

DLT messages not appearing after 3 retries

Verify `autoCreateTopics='false'` and the `.DLT` topic exists.

Homework, Checkpoint & What's Next



Daily Quiz — 8 questions, 10 minutes

🔗 Definition of Done — Session 13

- ☐ product-service-ci.yml triggers on push + PR
- ☐ Broken test → pipeline 🚫 (Quality Gate working)
- ☐ Image pushed to GHCR with sha- tag
- ☐ notification-service @RetryableTopic + DLT
- ☐ Commit: session-13: add-ci-pipeline...

🔗 Pre-Session 14 Reading

Topic: GitOps with ArgoCD + Kubernetes Deployment basics

- Push-based vs pull-based deployment?
- What is 'drift' in Kubernetes?
- What does a Deployment control?



SESSION 13 COMPLETE

CI Pipeline + Notification Service

Next: Session 14 — GitOps & Deployments: ArgoCD + Deployment Strategies

Monday, 3:00–5:30 PM, Online